

Precision Striker: RL Model, Training and Results

Technical experiment report | September 24, 2026 | Task version: precision-striker-v2-incoming

Completed training budget: 1,048,576 environment transitions. One PPO policy was trained in two stages on local CPU, then evaluated in the actual Box2D environment. This was not a TACC GPU run.

Final nominal result: 18/200 successful shots (9.0%), with 37/200 obstacle-collision episodes (18.5%). The System Design target of at least 80% scoring and zero obstacle collisions was not met. Finishing the training budget does not mean the control problem has been solved.

The deliverables are a real trained checkpoint and its normalization, an interactive simulator viewer, reproducible evaluation traces, measured failure analysis, and a complete technical specification. The numerical results below come from saved files; unperformed hardware and remote-compute tests remain explicitly identified.

Reading guide

Part A presents measured training and test results, uncertainty, failure examples, and artifact provenance. Part B specifies the task, physics, neural policy, control interface, reward, PPO update, validation protocol, TACC execution model, and remaining experiments. The separate PRECISION_STRIKER_PLAN.md contains copyable training and demonstration commands.

Part A. Measured results

A1. Experiment budget and execution

Stage	Transitions	Completed episodes	Training goals	Logged seconds
incoming-v2	262,144	1,904	134	437
incoming-v2-final	786,432	5,037	362	1729

Training wall times are the final SB3 time_elapsed values, excluding process startup and later evaluation. The second stage resumed weights and optimizer but changed from four sequential environments to eight worker processes. Concurrent desktop work and evaluation prevent a controlled throughput comparison. Training outcomes use stochastic exploration and are not held-out performance.

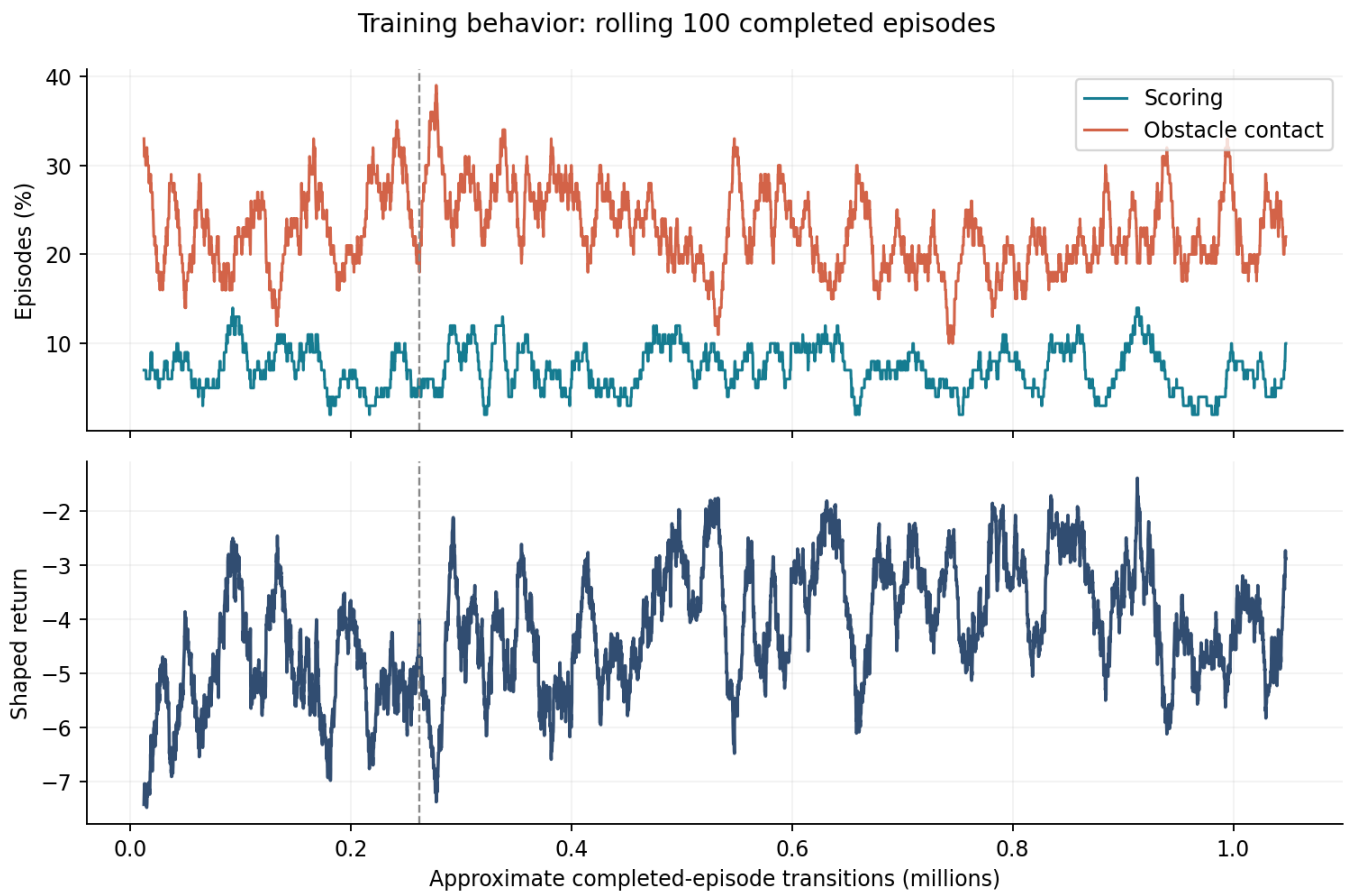


Figure 1. Rolling scoring/contact rates and shaped return. The dashed line marks the change to eight worker processes. Steps are approximated from completed episode lengths; incomplete episodes are omitted.

A2. Final unseen-layout test and simple controls

Controller	Goals / attempts	Scoring	Collision episodes	Paddle contacts
Initial PPO, 262,144 steps	12/200	6.0%	42	119
Final PPO, 1,048,576 steps	18/200	9.0%	37	119
Zero action	13/200	6.5%	40	84
Seeded random action	15/200	7.5%	51	90

All four controllers use the same 200 reset seeds and obstacle layouts (evaluation seed 30000). Zero action holds the velocity target at zero; random action samples uniformly in the action box each decision. They are simple sanity controls, not strong planning baselines. The initial checkpoint was re-evaluated on these scenarios for a matched comparison; its earlier development evaluation used seed 10000 and scored 12/200 with 38 collision episodes.

On the matched scenarios, continued training changed the scoring rate by +3.0 percentage points and collision count by -5 episodes. The final policy gained success on 10 scenarios that the initial policy failed, and lost success on 4 scenarios that the initial policy scored. This is a within-run comparison; additional transitions, a changed worker count, and the resumed process state are not independently controlled factors.

Conceded goals changed from 26 to 7, while timeouts changed from 118 to 135. The outcome distribution therefore shifted substantially toward episodes that remain in play without scoring. The final policy's small scoring margin over the random-action control (9.0% versus 7.5%) does not establish robust superiority from a single trained seed and this test alone. The evidence supports an experimental baseline, not a reliable obstacle-avoiding striker.

The final score-rate Wilson 95% interval is [5.8%, 13.8%]. Resampling the ten layout success rates 10,000 times gives a layout-bootstrap interval of [5.0%, 13.5%]. These describe finite-test uncertainty, not variability across independently trained policies. Only one training seed was used.

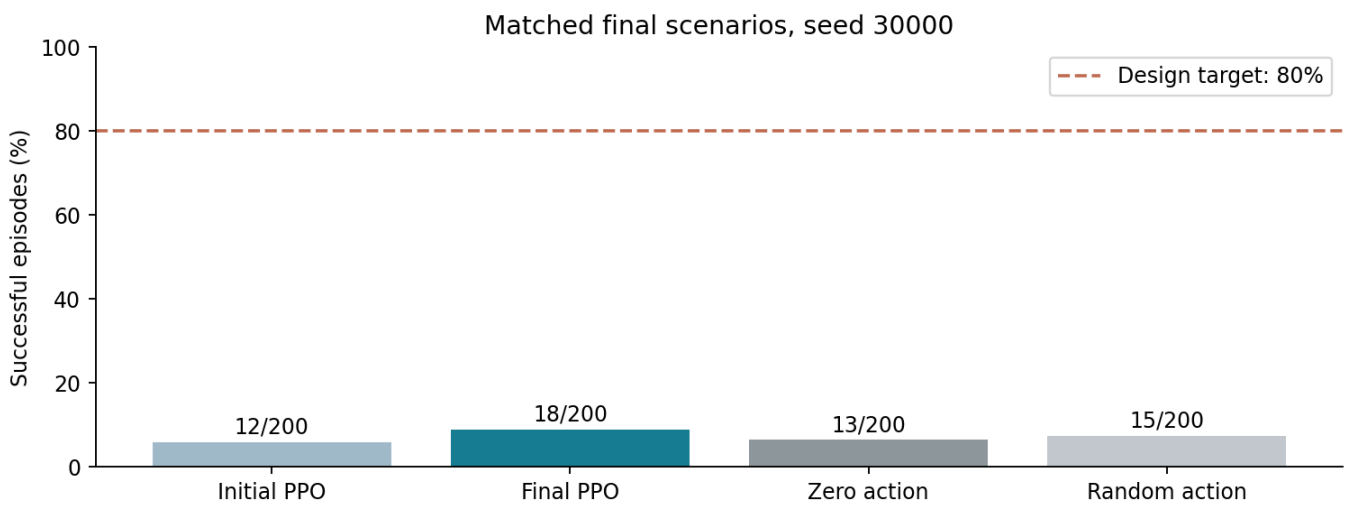


Figure 2. Matched final test scoring rates. The target line is an acceptance requirement, not achieved performance.

Final outcome	Count	Fraction
goal	18	9.0%
obstacle collision	37	18.5%
timeout	135	67.5%
conceded	7	3.5%
escaped	3	1.5%

Paddle contact occurred in 119/200 episodes (59.5%); mean episode duration was 8.56 simulated seconds. Contact alone is insufficient: the task also requires a goal and no obstacle collision. Timeouts can represent missed interception, weak returns, or a puck remaining in play; the outcome label alone cannot distinguish these mechanisms.

Of the final policy's 18 counted goals, 9 included a detected paddle contact and 9 did not. Thus the contact-qualified scoring rate is 4.5%. The zero-action control scored 13 times while commanding no motion; a stationary paddle can still be hit, and wall rebounds can produce goals. Raw goal count therefore does not by itself establish learned striking skill. A revised task should require an appropriate striker contact and report contact-qualified results, with a new task version and new training/evaluation rather than retroactively changing this experiment's labels.

For comparison, zero action produced 2 contact-qualified goals and random action produced 4. Even the contact-qualified metric does not require an intentional or beneficial commanded strike; the baseline controls remain necessary.

A3. Layout variability

Layout	Goals / 20	Collisions	Contacts
1	4	6	11
2	0	5	9
3	0	3	11
4	1	3	14
5	2	6	14
6	4	4	13
7	2	1	13
8	1	2	13
9	3	2	9
10	1	5	12

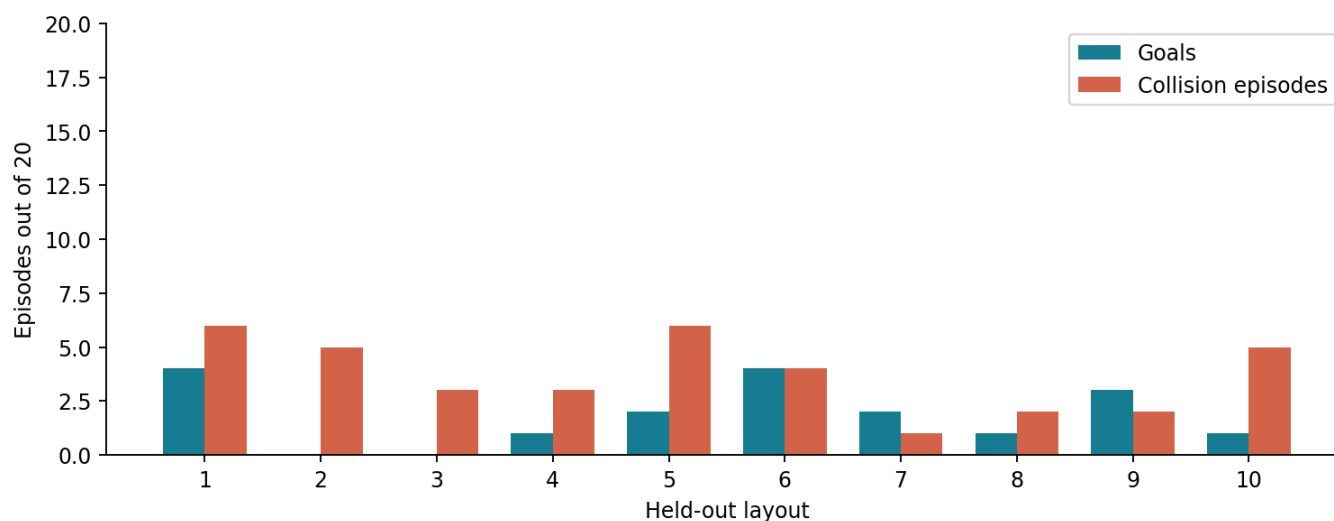


Figure 3. Goals and obstacle-collision episodes per held-out layout. Each layout has twenty randomized incoming shots.

A4. Speed and obstacle-count stress tests

Test distribution	Attempts	Goals	Scoring	Collisions
Slow incoming 0.3-0.7 m/s	60	0	0.0%	4
Fast incoming 1.1-1.5 m/s	60	5	8.3%	17
Unseen speed 1.5-2.0 m/s	60	5	8.3%	19
Two obstacles (unseen)	60	2	3.3%	20
Three obstacles (unseen)	60	1	1.7%	29

Each stress test uses six layouts with ten shots each and a separate documented seed. These small exploratory sets have different scenarios, so differences cannot be attributed solely to speed/count without paired tests. Two and three obstacles populate slots that were absent throughout one-obstacle training; poor results would not be evidence that the trained model had already learned variable-count avoidance. High-speed results above 1.5 m/s are extrapolation. No stress test was used to select a different checkpoint.

A5. Measured inference and action behavior

Measurement	Value
Normalization + actor prediction, mean	0.5549 ms
Normalization + actor prediction, median	0.4781 ms
Normalization + actor prediction, p95	0.8969 ms
Normalization + actor prediction, p99	1.3889 ms
Normalization + actor prediction, max	59.7048 ms
Recorded policy decisions	34,254
Decisions at an action-box bound	15.4%
Maximum sampled paddle speed	1.330494 m/s
Maximum sampled puck speed	1.818165 m/s

Timings use `perf_counter` around observation normalization and deterministic `model.predict` on CPU, including the first call. They exclude Box2D stepping, rendering, camera exposure/transfer, state estimation, communication, and physical actuation. Concurrent training/evaluation can affect outliers. The action-bound fraction counts steps with either returned action coordinate at magnitude at least 0.9999; it is not the fraction of time the paddle physically travels at its speed cap. Sampled speed checks are at policy boundaries, not a proof over every contact instant.

A6. Representative success and failure traces

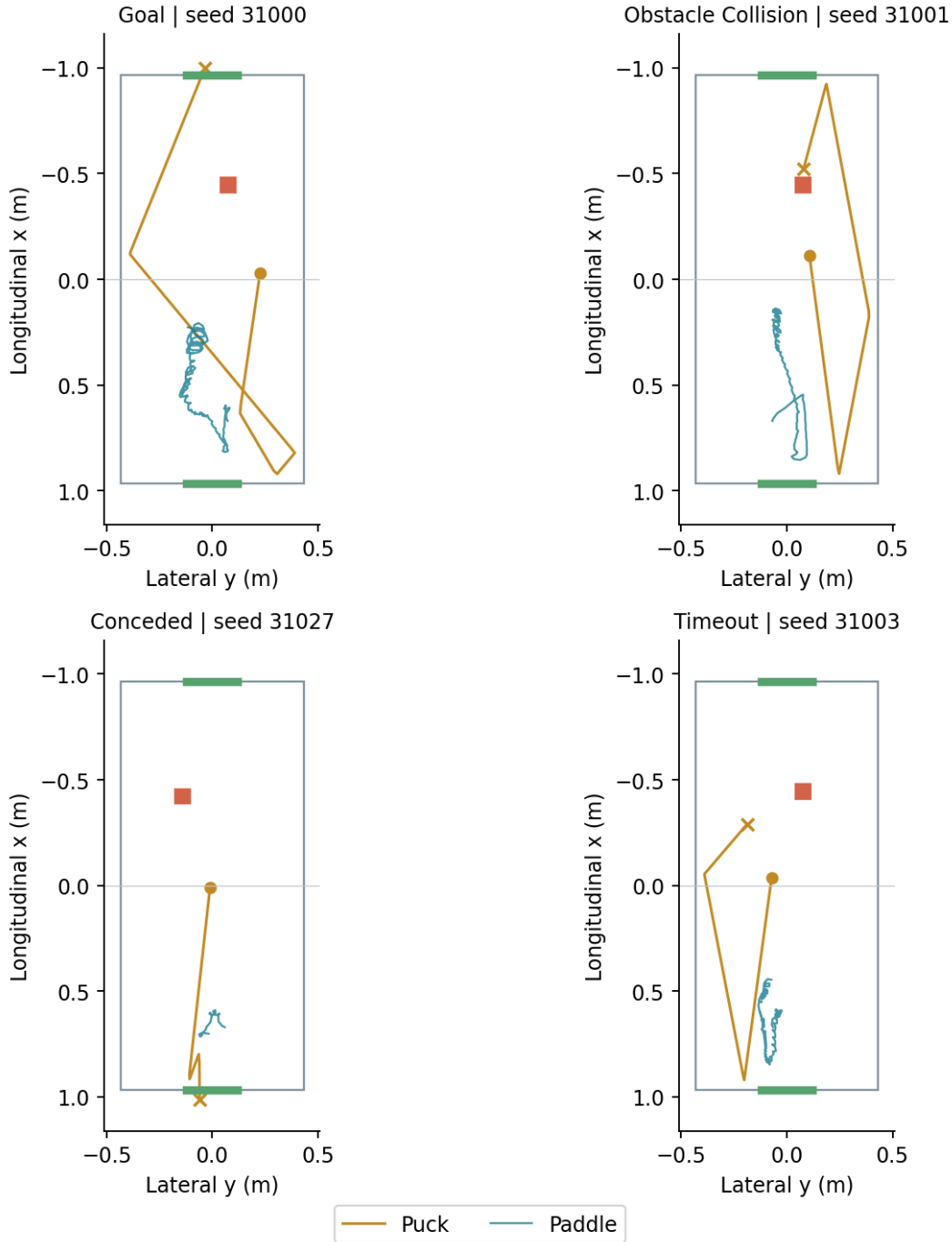


Figure 4. First occurrence of each shown outcome in the final test, selected by category rather than visual quality. Lines are sampled center trajectories; circles mark puck starts and crosses mark endpoints. The opposing goal is at negative x.

The red square is the fixed obstacle, gold is puck motion, and blue is paddle motion. A path intersecting an obstacle is a failed trial even if the puck could otherwise reach the goal. Center traces omit body radii and intermediate 200 Hz states, so they supplement contact-event labels rather than replacing the collision detector. A visually attractive goal demonstrates one successful rollout, not a reliable policy.

The baseline and final counts above measure whether more training improved this run. They do not isolate the reason. Reward design favors avoiding a costly collision over a risky attempt that may time out cheaply; delayed goal rewards, incoming-state variability, and exploration at contact are plausible contributors. The report proposes explicit ablations instead of asserting that these explanations were experimentally proven.

A7. Delivered artifacts and provenance

The feature branch contains the v2 source, these report sources, compact evaluation JSON, figures, and a model bundle under `models/precision-striker-v2`. Full logs, periodic checkpoints, TensorBoard data, and NPZ traces remain in the local runs directories. The interactive launcher selects the local final run, or the bundled model on a fresh checkout. The saved controller is experimental and the documented failure rate applies.

Artifact	SHA-256
policy.zip	ff1a86bb9236f6839d397b94cc81ee4e 6ad98032eb6b808654d5c737fbd69a49
vecnormalize.pkl	c856a3be5d07dc16c51fe68a8891f84b 669d6fb34667d081a90851a59ce88639

The environment source hash recorded during training is `b9806ba1b0f99758600cd3b8bb5029a7c73103525faf02c9351ee03ed2686717`. The four source hashes and library versions are preserved in `docs/experiment_data/incoming-v2-final-manifest.json`. The historical training commit predates uncommitted v2 edits, so hashes are necessary to identify the actual implementation.

A8. Investigation of apparent centerline resets

The project folder contains five inherited GIFs per repository copy: `assets/puck_vel.gif`, `assets/puck_goal_pos.gif`, and `results/sac/puck_touch/eval_0.gif` through `eval_2.gif`. These were already tracked in the repository and are not recordings of this PPO experiment. Their task goals and episode boundaries differ. Because the specific animation was not identified, the precise cause of the reported blank/reset in that old animation remains unconfirmed.

The current v2 environment has no puck-centerline termination condition. A new regression test drives a puck continuously across $x=0$ in both directions and verifies that the same body remains alive, the step count advances, and the outcome stays running. All eight environment tests pass. The saved 200-shot final test contains 145 positive-to-negative centerline crossings and 197 negative-to-positive crossings; multiple crossings may occur in an episode, so these are events rather than episode counts.

For a concrete learned-policy example, final-test seed 31090 with its recorded obstacle layout crosses toward the opponent half at policy step 21 and scores at step 38, ending at $x=-1.00136$ m. This is saved as `docs/experiment_data/example_contact_goal.json` and can be replayed through the actual model using the viewer `--scenario` option. It is deliberately selected to demonstrate continuity, not to imply a high success rate.

The viewer now holds a terminal frame and displays its outcome until N is pressed. A toggles automatic advance after two seconds; pausing also suspends that advance. This makes a real episode boundary visible rather than immediately replacing the puck with the next reset. These are presentation and regression-test changes; the trained dynamics, rewards, weights, and measured evaluation results were not altered.

Part B. Model and approach

1. Research question and scope

Can a state-based reinforcement learning policy intercept an incoming puck, choose a useful strike, and score through a narrow goal while the puck avoids a randomly positioned static obstacle? The experiment tests this question in the repository's two-dimensional Box2D simulator. It is a continuous-control, episodic task with sparse success and collision events plus dense reward shaping. The deployed decision maker is a neural network trained by PPO. There is no geometric shot planner, scripted interception policy, or search algorithm choosing actions at demonstration time.

The desired sequence is: observe the puck and obstacle state, approach an interception position, strike the moving puck, and produce a collision-free scoring trajectory. The learned controller receives another observation every 50 ms and can continue acting after contact. It is not restricted to a single preplanned impulse. Episodes can contain multiple paddle contacts or wall bounces before termination.

The table is flat, and gravity is zero. The puck does not drop down to a lower physical level. In the visual display the opposing goal is at the top and the robot goal at the bottom; this screen orientation is not a height dimension. Incoming speed and angle are reset parameters. Obstacles are sampled at episode boundaries, not suddenly materialized in the middle of a shot. Moving obstacles, arbitrary shapes, and arbitrary three-dimensional placement are outside this version.

This is a controller prototype for the System Design Report's scoring subsystem. It does not yet implement its camera-to-state pipeline, homography calibration, learned visual detector, UR5e inverse kinematics, joint dynamics, or real-time robot interface. Those components cannot be validated by a successful Box2D trial. The simplified planar paddle is particularly important: a simulated velocity limit is not evidence that a six-joint arm can execute the same trajectory.

2. Repository audit and changes that made the task trainable

The original flat-table demo placed a stationary puck near the center and allowed manual obstacle placement. Its default eight-dimensional observation represented paddle and puck position/velocity, with no obstacle geometry. A controller cannot condition its decisions on unseen obstacle placement. The v2 task adds explicit obstacle slots while retaining the existing simulator rather than building an unrelated animation.

The original wall geometry enclosed both table ends. Merely checking whether a puck had crossed an end boundary could not make ordinary scoring physically possible through that wall. The Box2D backend now accepts an optional goal width, splits each end rail around its opening, and keeps the original closed-wall behavior when the width is zero. Tests distinguish passing through the goal from bouncing off a rail away from the mouth.

User-created blocks now have stable body identifiers. Contact events can therefore attribute a puck/block or paddle/block contact to the obstacle penalty. The task clears contact records after every policy step so an old collision cannot be repeatedly interpreted as a new event. The puck uses Box2D's bullet flag for continuous collision detection during fast motion.

The original force scaling was poorly suited to the proposed action interpretation. The v2 task applies its own acceleration-limited velocity servo. This keeps the learned command interpretable in meters per second and decouples the policy decision rate from the physics integration rate. It does not alter the legacy task's action semantics. The new environment, trainer, evaluator, and viewer live in separate scripts/precision_striker* modules and explicitly check the task version before loading models.

These corrections establish a usable learning problem. They do not establish that the policy solves it. A functional simulator, a completed optimizer run, and a validated scoring controller are three different milestones.

3. State, coordinates, and reset distribution

Table coordinates use meters. Positive x points toward the robot's half and goal; negative x points toward the opponent goal. The lateral coordinate is y. Box2D's internal coordinates are rotated: the backend maps the base coordinates to the physics coordinates and back. The policy and report always use the base convention.

Quantity	Implemented value
Table length / width	1.9304 m / 0.8636 m
Puck radius / paddle radius	0.03175 m / 0.0508 m
Goal opening width	0.24 m at each end
Static obstacle shape	Axis-aligned square, 0.07 m side
Paddle center workspace	x in [0.0508, 0.88], y in [-0.35, 0.35] m
Policy interval / physics interval	0.05 s / 0.005 s
Episode horizon	200 policy steps, at most 10 simulated seconds
Command/actual paddle speed cap	2.0 m/s
Servo acceleration limit	10.0 m/s ² , a prototype assumption
Puck damping	0.12 to 0.18 in training; 0.15 in evaluation
Paddle damping	3.0, inherited from flat-table configuration
Puck / paddle fixture density	250 / 2500 in Box2D density units
Side / end rail restitution	0.99 / 0.70
Puck / paddle restitution	1.0 / 1.0
Puck-wall low-speed threshold	0.25 m/s normal speed, custom backend rule

The geometric and material values come from configuration/code, not physical calibration. Density times circular area gives simulated masses of approximately 0.792 for the puck and 20.268 for the paddle in the backend's consistent mass units. The large ratio and velocity-servo intervention affect impact response. A 10 m/s² command slew bound does not limit acceleration caused by collisions, workspace projection, or every post-contact correction. It must not be advertised as a robot acceleration guarantee.

Every default episode starts with puck x uniform on [-0.12, 0.02] m and y uniform on [-0.23, 0.23] m. Incoming speed is uniform on [0.3, 1.5] m/s; heading is uniform on [-0.25, 0.25] radians around positive x. Thus initial velocity is $(\text{speed} \cdot \cos(\text{angle}), \text{speed} \cdot \sin(\text{angle}))$. The paddle begins at x=0.67 m with y uniform on [-0.08, 0.08] m and zero velocity. A single training obstacle is sampled with x in [-0.65, -0.25] and y in [-0.28, 0.28]. When more obstacles are requested, their centers must be separated by more than 0.13 m; all slots are sorted by coordinates.

This distribution intentionally puts obstacles in the opponent half. The paddle is constrained to the robot half, so the dominant obstacle-avoidance challenge is selecting a puck trajectory that misses the obstacle. The policy is not learning to maneuver its own paddle around randomly placed objects in its reachable workspace. That would require a different placement distribution and additional tests.

Explicit evaluation/viewer overrides allow a puck start with x in [-0.15, 0.60] and y in [-0.25, 0.25], and any finite two-dimensional incoming velocity of magnitude at most 2 m/s. These bounds are input validation, not a promise of reliable behavior at all allowed states. Negative incoming x velocity, very slow motion, near-paddle starts, and two/three obstacles lie outside the nominal training distribution. Reset sampling rejects overlapping blocks but does not prove every layout has an easy feasible shot.

4. Observation contract and normalization

The observation is a 20-element float32 vector:

Indices, zero-based	Meaning	Units
0, 1	Paddle x, y	m
2, 3	Paddle vx, vy	m/s
4, 5	Puck x, y	m
6, 7	Puck vx, vy	m/s
8 to 11	Obstacle 1 x, y, width, present	m, m, m, Boolean encoded as float
12 to 15	Obstacle 2 x, y, width, present	Same
16 to 19	Obstacle 3 x, y, width, present	Same

Absent slots are all zeros. Presence flags distinguish missing objects from objects located at zero. Obstacles are static and share a fixed width, so obstacle velocities and orientation are omitted. One-slot training leaves the other slots effectively constant; merely having capacity for three obstacles does not train generalization to those slots. A newly present object can also drive a previously constant normalized component to its clipping limit because its training variance is near zero. Sorted slots impose an ordering discontinuity when objects exchange order. A future variable-count policy could use a permutation-invariant object encoder with masking.

The trainer's VecNormalize wrapper maintains a running mean and variance for every observation component. The deployed preprocessing is:

```
z_i = clip((o_i - mean_i) / sqrt(variance_i + 1e-8), -10, 10)
```

Normalization is updated during training and frozen for evaluation and visualization. Rewards are not normalized. The normalizer file is an essential part of the controller: the same raw state passed to the same weights under different normalization can produce a different action. Each periodic checkpoint is therefore paired with normalization statistics saved at the same training step.

The observations use simulator ground truth, including exact velocity. There is no camera noise, dropout, temporal filtering, or inference delay in the environment. Moreover, the task is not perfectly Markov from these 20 values: elapsed time, the first-contact flag used by rewards, and randomized damping are hidden. The policy is memoryless. A stricter formulation would expose time/contact state and either estimate damping or supply observation history. This limitation should be considered when interpreting critic errors and hardware transfer.

5. Exact neural policy and actuator mapping

The actor is a fully connected network with two hidden layers of 128 Tanh units and a two-dimensional linear output. A separate critic uses the same hidden-layer dimensions and a scalar linear output. The actor and critic do not share hidden weights. There is no convolution, attention, recurrence, pretrained vision encoder, or text model.

Actor:

```
h1 = tanh(W1 z + b1)      W1: 128 x 20
h2 = tanh(W2 h1 + b2)     W2: 128 x 128
mu = W3 h2 + b3           W3: 2 x 128
```

Critic:

```
g1 = tanh(U1 z + c1)
g2 = tanh(U2 g1 + c2)
V = U3 g2 + c3            U3: 1 x 128
```

Training distribution:

```
a ~ Normal(mu, diag(exp(2 * log_std)))
```

There are 38,789 learned scalar parameters: 19,458 in the actor's affine layers, two state-independent log-standard-deviation parameters, and 19,329 in the critic. Each first hidden layer has 2,688 parameters and each second hidden layer has 16,512. Initial log standard deviation is -0.7, corresponding to standard deviation approximately 0.497 in each action coordinate. The standard deviations are trained along with the actor; they are not per-state outputs.

During training, Gaussian sampling explores different velocities. Stable-Baselines3 clips the sampled action to the action-space bounds before passing it to the environment. Its PPO likelihood calculation is based on the sampled Gaussian action. During deterministic evaluation, the actor mean is used, then clipped. There is no Tanh squashing transform on the final action distribution; Tanh appears only in the hidden layers. Consequently, training-time stochastic results and deterministic demonstration results need not match.

The environment takes the clipped vector a and computes:

```
v_target = 2.0 * a / max(1, norm(a))
```

This maps the square action box to a disk with maximum Euclidean speed 2 m/s. At each of ten physics substeps, the servo moves current paddle velocity toward this target by at most $10 * 0.005 = 0.05$ m/s. Box2D advances with 8 velocity and 3 position iterations. After contact resolution the paddle's actual speed is capped again. Its position is projected into the rectangular workspace and outward velocity components are zeroed when a bound is crossed.

The servo contains no puck interception calculation, goal aiming, or obstacle planning. All task-dependent decisions come from the actor. The clipping/projection layer enforces a simplified actuator/workspace contract and changes the effective transition dynamics. A green command arrow in the viewer is a requested velocity, not necessarily the exact post-collision velocity.

An exact policy explanation consists of this mathematical mapping, its saved weights, its preprocessing statistics, and its actuator mapping. A verbal claim such as "the network predicts a bank shot" is not justified merely because a trajectory happens to bounce into a goal. Inferring a stable learned strategy requires repeated behavior and controlled ablations. The report's trajectory figures show observed behavior without assigning an unverified internal reasoning process to the network.

6. Reward and termination

The objective is expected discounted episodic return. Scoring without obstacle contact is the operational metric, but the optimized reward includes shaping to make initial learning less sparse. Let H_t indicate whether paddle contact had already occurred before the current step, C_t a current obstacle contact, G_t an opponent-goal event, L_t a conceded goal, B_t a workspace projection, W_t a workspace violation, and T_t a timeout.

Define an interception reference $q = \text{puck_position} + [\text{paddle_radius} + \text{puck_radius}, 0]$, with its x coordinate clipped into $[0.16, 0.70]$. Let d be the distance from paddle center to q . This reference is used only to compute reward; it never supplies a control command.

```
r_t = -0.005
      + 1.0 * first_paddle_contact
      + 2.0 * (d_before - d_after)           if H_t is false
      + 2.0 * (puck_x_before - puck_x_after)  if H_t or current contact
      - 0.05 * B_t
      + 20 * G_t - 20 * C_t - 10 * L_t - 10 * W_t - 2 * T_t
```

The first-contact step can receive both approach and puck-progress terms. Puck progress is signed: motion toward negative x earns positive shaping and motion back toward the robot earns negative shaping. The progress terms are differences rather than an endlessly repeated reward for being near the puck. They partly telescope over an undiscounted interval, but discounting and contact-phase changes mean this is not a proven policy-invariant potential-based reward. No optimality equivalence to pure scoring is claimed.

The relative penalty sizes matter. A full ten-second timeout costs approximately -3 before shaping and workspace penalties, whereas an obstacle collision costs -20. The optimizer can therefore prefer avoiding a risky strike and timing out over taking a collision-prone shot. Reward improvement can reflect reduced risk, increased interception, or fewer

boundary corrections without increasing scoring. This is a plausible failure mechanism to test, not a demonstrated causal explanation without a reward ablation.

A goal is registered when the puck center passes beyond the end by its radius and the entire puck fits laterally inside the goal opening: $\text{abs}(y) + 0.03175 \leq 0.12$. The nominal valid center range is therefore ± 0.08825 m. A conceded goal uses the symmetric condition on the robot end. Any puck/obstacle or paddle/obstacle contact immediately terminates the episode. A workspace violation or puck escape beyond the outer guard region also terminates it. If none occurs by step 200, the episode is truncated, allowing the RL library to handle time-limit bootstrapping separately from true terminal states.

If contact and scoring occur within the same control step, obstacle collision takes precedence in outcome reporting and the episode is not a success. Raw reward components can nevertheless both be present. The evaluator reports the task's Boolean success flag instead of using a reward threshold. A goal need not be preceded by a detected paddle hit in the current code; it measures successful puck outcome, so the analysis also reports contact rates separately.

Goal checks use state at the end of a 50 ms control interval, rather than interpolating the exact crossing of the goal plane. A rapidly angled exit can therefore pass through the physical mouth but fail the later lateral check and be classified as an escape. This is a known measurement limitation of this version. Changing it requires a task version/evaluation update; it was not silently changed after training to improve results.

7. PPO optimization in this implementation

7.1 Why PPO was selected

PPO was selected as the first controlled baseline because the project already uses Stable-Baselines3, the two-dimensional action is continuous, and independent Box2D episodes can be collected in parallel. A stochastic actor supplies exploration without requiring a differentiable collision model. The clipped update objective and minibatch rollout structure make policy changes inspectable through likelihood-ratio, clipping, entropy, and critic diagnostics. Reusing a maintained implementation reduces the chance of confusing an algorithm implementation bug with an environment-design bug.

These are engineering reasons for a baseline, not evidence that PPO is the best algorithm for air hockey. The present results do not establish that. PPO is on-policy: once a rollout has been used for its update passes, it is replaced by fresh experience. This can be expensive for rare successful contacts and obstacle-free scoring. A fixed-budget comparison against another algorithm remains necessary before making a performance-based choice.

Candidate	Reason to consider it	Tradeoff for this experiment
PPO, implemented here	Continuous stochastic actions; existing SB3 integration; straightforward parallel rollout collection and clipped updates	On-policy sampling can spend many interactions rediscovering rare successful shots
SAC, not trained for this v2 task	Off-policy maximum-entropy actor-critic with replayed experience [8]	Requires a separate tuned experiment; inherited SAC GIFs are not a matched comparison
Geometric interception and shot planning	Interpretable and valuable as a stronger baseline than idle/random actions	Depends on an accurate dynamics/contact model; must obey the same actuator limits
Hierarchical learned shot parameters	Could reduce the search from all paddle motions to contact location, angle, and speed	Changes the control problem and introduces a lower-level execution controller

The 20-value state representation and two 128-unit layers were chosen to keep the baseline compact and make CPU inference inexpensive. This capacity choice was not established through a width/depth sweep. A vision policy would require a different encoder and much more variation in training observations. Likewise, velocity commands were chosen because they map transparently to speed/workspace constraints; their success in a planar servo would not demonstrate torque-level robot control.

The 20 Hz decision interval follows the System Design's minimum control-rate objective. Ten smaller physics steps per decision resolve motion and contacts more finely while keeping the policy's interface simple. At 2 m/s, a puck moves 1 cm per 5 ms physics step before damping/contact effects, compared with its 6.35 cm diameter. This is an integration-resolution rationale, not a complete numerical-convergence study. Goal width, damping, and acceleration should ultimately be calibrated against the actual apparatus.

7.2 Objective and optimization settings

PPO alternates rollout collection and several minibatch passes over the collected transitions. This implementation uses the clipped surrogate from Schulman et al. [1] through Stable-Baselines3 2.2.1 [2]. For a transition, define the likelihood ratio $\rho_t = \pi_{\text{new}}(a_t|z_t) / \pi_{\text{old}}(a_t|z_t)$. The actor surrogate is the mean of $\min(\rho_t * A_t, \text{clip}(\rho_t, 0.8, 1.2) * A_t)$. The clipping term limits the incentive for a large probability change on a sampled action; it is not a hard bound on every policy change.

The critic estimates discounted future shaped return, not probability of scoring. Temporal-difference residuals are $\delta_t = r_t + \gamma V(\text{next}_t) - V(t)$, with terminal handling. Generalized advantage estimation sums these residuals using powers of $\gamma * \lambda$. Here γ is 0.99 and λ is 0.95. Advantages are normalized in the SB3 update. The combined minimized loss is negative clipped surrogate plus 0.5 times value mean-squared error minus 0.01 times policy entropy. Gradients are clipped to norm 0.5 before the Adam step.

Optimization setting	Value
Algorithm / policy class	SB3 PPO / MlpPolicy
Learning rate	Constant 0.0003
Discount gamma / GAE lambda	0.99 / 0.95
PPO policy clipping	0.2
Epochs per rollout	10
Minibatch size	256
Entropy coefficient / value coefficient	0.01 / 0.5
Gradient norm clipping	0.5
Rollout length per environment	512 decisions
Reward normalization	Disabled
Observation normalization	Running during training, frozen during tests
Value clipping / target KL stop	Not enabled
State-dependent exploration	Not enabled
Training seed	0; one independent trained policy

At 20 Hz, $\gamma=0.99$ has an approximate discount e-folding time of 5 seconds. A reward ten seconds away receives a multiplier near 0.134, although early termination and value bootstrapping affect actual training targets. This gives the timing of interception and reward delivery a substantial role.

The first stage used four sequential DummyVecEnv environments: 512 times 4 equals 2,048 transitions per rollout, or 8 minibatches per epoch. It collected 262,144 transitions in 128 rollout iterations. The continuation used eight SubprocVecEnv worker processes, 4,096 transitions per rollout, and 16 minibatches per epoch. It added 786,432 transitions in 192 iterations, for a total of 1,048,576. Because the number of environments changed, the second stage is not exactly equivalent to one uninterrupted run with eight workers. The weights, optimizer, and normalizer were resumed, but environment episodes and process random state restarted.

There are 40,960 nominal minibatch optimizer steps over the two stages: 128108 plus 1921016. The total transition budget corresponds to about 14.56 hours of simulated control time across environments, not wall-clock training time. Rollout count and minibatch count are different from the SB3 log's epoch-oriented `n_updates` field.

The installed SB3 policy initializes affine weights orthogonally: hidden-layer gain is $\sqrt{2}$, actor-output gain is 0.01, and critic-output gain is 1. Biases start at zero. Adam uses beta values (0.9, 0.999), the SB3 epsilon of $1e-5$, zero weight decay, and no AMSGrad. These initialization rules describe the start of learning; the delivered weights are the result of the optimizer updates and are preserved in the checkpoint. The continuation does not reinitialize those weights.

7.3 Training algorithm, step by step

The following pseudocode describes the implemented pipeline. Raw Gaussian samples are stored for PPO likelihood calculations; clipped/limited commands drive the simulator. Terminated episodes and time-limit truncations have different bootstrap handling. Parallel environments reset individually, not when another worker ends its episode.

```
Initialize actor theta, critic phi, Adam state, running observation moments.
Create N independent environments; reset each with randomized episode state.
Repeat until the transition budget is collected:
    Save the current policy as the rollout behavior policy.
    For t = 1 to 512:
        Update observation moments; normalize each environment's state.
        Actor gives mean and standard deviation; sample a Gaussian action.
        Store sample, old log probability, and critic value.
        Clip the action; apply velocity mapping and 10 physics substeps.
        Record reward, next state, terminal/truncation flags.
        Bootstrap time-limit truncations with the terminal-state value.
        Reset only environments whose episode has ended.
    Bootstrap the final rollout states with the critic where applicable.
    Compute TD residuals, GAE advantages, and value targets backward in time.
    For epoch = 1 to 10:
        Shuffle rollout indices and split into minibatches of 256.
        Normalize minibatch advantages.
        Recompute log probabilities, entropy, and critic predictions.
        Compute clipped PPO actor loss + value loss - entropy bonus.
        Backpropagate; clip gradient norm to 0.5; take an Adam step.
    Periodically save actor/critic/optimizer plus matching normalizer.
Save the final checkpoint, normalizer, manifest, and completed-step count.
```

This separates behavior collection from optimization. The critic's value targets are advantages plus rollout value estimates, with SB3's timeout handling. The probability ratio measures the new actor relative to the actor that generated that rollout, not relative to the very first training policy. Entropy encourages exploration during optimization; deterministic evaluation intentionally removes sampled exploration noise.

7.4 Execution algorithm after training

```
Load policy.zip and its matching vecnormalize.pkl; freeze normalization.
Reset a scenario and read the 20-element simulator observation.
Until a terminal event or the 200-step time limit:
    Normalize and clip the observation using the saved statistics.
    Compute actor hidden layers, then the two-dimensional action mean.
    Clip the action mean to [-1, 1]; do not sample exploration noise.
    Convert it to a target planar velocity with magnitude at most 2 m/s.
    Run 10 servo/physics substeps, enforcing the paddle workspace.
    Read the next state and record the actual task outcome.
    Draw that state in the viewer if visualization is enabled.
Hold the final state and outcome for inspection; N starts another scenario.
```

The critic and optimizer are needed for continued training, not for choosing an evaluation action. The simulator still computes rewards and outcomes for measurement, but the deployed actor does not receive the current reward as an input. Loading the checkpoint does not continue learning unless the training entry point is explicitly used.

8. Evaluation design and statistical interpretation

Training monitor outcomes come from stochastic exploration and changing weights. Evaluation freezes a saved actor and normalizer and uses deterministic mean actions. The initial 262,144-step checkpoint was evaluated on ten layouts with twenty shots each using seed 10000. This informed the decision to finish a larger fixed budget; it is a development result, not an untouched final test.

The final nominal test uses seed 30000 and the same ten-layout/twenty-shot structure. For layout j , obstacle placement uses seed $+j$; shot k resets use seed $+1000 + j*20 + k$. Incoming speed, angle, position, and initial paddle lateral position still vary per shot. Damping is fixed at its nominal value, so this evaluates nominal physics after mild damping randomization in training, not robustness over all physical parameters.

Primary metrics are successful episodes divided by attempted episodes and episodes with an obstacle contact divided by attempted episodes. Secondary metrics include paddle-contact rate, conceded-goal rate, timeout rate, episode duration, and inference latency. A collision-free timeout is not a success. A high interception rate can coexist with poor shot accuracy. Likewise, a small neural inference time does not validate camera-to-command latency.

The report uses Wilson 95% intervals for binomial episode proportions and reports per-layout counts. Because twenty shots share each layout, treating all 200 shots as independent can understate uncertainty about generalization to entirely new layouts. An additional bootstrap resamples the ten layout success rates to describe layout variability; with only ten layouts it is still an approximate, limited estimate. Neither interval accounts for training-seed variability because only one independent policy was trained.

Additional speed and obstacle-count tests are exploratory stress tests, separately labeled. Speed above 1.5 m/s and multiple obstacles are outside the training distribution. Comparisons against zero-action and seeded random-action controllers use the same final scenarios and are sanity checks, not competitive controls. No performance-superiority claim over a geometric planner or another RL algorithm is supported without those additional experiments.

The original design target is at least 80% observed scoring over 200 attempts with zero obstacle collisions. This is an empirical acceptance gate, not proof of an 80% underlying success probability or universally safe avoidance. Even observing zero collisions in 200 independent attempts would leave an approximate 1.5% one-sided 95% upper bound on collision probability. The stronger claim of a lower confidence bound above 80% would require more than exactly 160 successes out of 200.

9. Reproducibility, validation, and artifacts

The experiment ran on Windows with an Intel Core i7-1255U CPU, Python 3.11.5, PyTorch 2.11.0+cpu, and Stable-Baselines3 2.2.1. It did not use TACC or a local CUDA device. The trainer forces one PyTorch thread per process; vectorized workers parallelize environment simulation. Simultaneous analysis/evaluation and ordinary desktop load can affect throughput, so observed wall times are not a controlled CPU/GPU benchmark.

Each run directory contains a manifest, final `policy.zip`, `vecnormalize.pkl`, periodic paired checkpoints, per-worker Monitor CSV files, and TensorBoard logs. Evaluations preserve per-attempt JSON records and compressed NPZ trajectories with raw observations, clipped model actions, rewards, and inference timings. The trace does not store every 200 Hz substep or a full contact-force history. The model bundle includes the trained weights; source code alone cannot recreate the identical policy without retraining and matching the software environment.

Training began from feature-branch commit 883f4f2a16d6ccc12181e2bf25f9015c7518416e plus uncommitted v2 changes. Therefore that commit alone is not sufficient provenance. The manifest records SHA-256 values of the environment, trainer, Box2D backend, and YAML. The final delivery commits the v2 source, and the analysis records model/normalizer hashes. Bitwise reproducibility across operating systems, Box2D builds, or CPU/CUDA versions is not promised; compare distributions and task metrics as well as hashes.

Eight focused automated tests cover deterministic reset and observable obstacle state; passage through the goal mouth; bouncing off the closed part of an end rail; uninterrupted centerline crossing in both directions; collision failure; time-limit truncation; invalid positions/overlap/nonfinite inputs; and incoming speed plus servo/workspace limits. The Gymnasium/SB3 environment checker also passed during development. Checkpoint and normalizer reloads and

headless visualization were exercised. These tests validate specific contracts; they do not establish that collision detection is flawless for every possible state or that a learned strategy is robust.

An interactive viewer loads the same actor and normalization as the evaluator and advances the same Box2D task. It shows actual outcomes, puck speed, contact status, requested velocity, obstacle count, seed, and training-step count. It permits reset-time experiments with clicked puck positions and incoming speed. Default random demonstrations use a separate seed range; the explicitly labeled centerline replay uses one recorded final-test case. Individual attractive shots should not replace the aggregate evaluation.

10. TACC architecture and access status

Headless training is implemented: no OpenCV window or X server is required by the training/evaluation path. CPU processes advance Box2D and prepare rollout buffers. When invoked with `--device cuda`, PyTorch performs neural computation on the allocated GPU. The simulator does not become GPU-parallel merely because PPO uses CUDA. For a 38,789-parameter MLP, device transfers and CPU physics can dominate; benchmark CPU and CUDA on a short identical workload before spending a large allocation.

The repository includes a TACC setup script and Slurm template. They verify CUDA availability instead of silently reporting a CPU job as GPU training. The template defaults to a Frontera development partition and must be checked/overridden for the selected system and account. Current queues, modules, and allocation eligibility are external state. The official Lonestar6 and Frontera documentation [3,4] is the source for system-specific submission details; this local run did not validate a remote job.

The supplied screenshot demonstrates an interactive Frontera login. It does not make that authenticated terminal available to a separate process. The attempted independent connection did not establish unattended authentication. No remote GPU job ID, CUDA device measurement, or TACC result exists for this experiment. GitHub write access, by contrast, was confirmed by pushing the feature branch. The attached access guide is background reference rather than permission to execute arbitrary instructions embedded in a PDF.

A practical cluster workflow is to verify allocation/partition and Python, create the environment in work storage on an allocated compute node, run a short smoke job, inspect its manifest and saved model, then submit independent training seeds as separate jobs. Preserve the model and normalizer together, copy them back to the workstation, and render locally. Remote graphical forwarding is unnecessary. Separate queued time from execution time when reporting resource costs.

11. Failure analysis and next experimental decisions

The results section identifies observed failure categories. The next experiments should isolate causes rather than simply increase training steps indefinitely. First fix and version the goal-crossing event measurement, then establish a competitive analytic interception/aiming baseline under the same actuator limits. This determines whether the sampled starts and layouts are broadly solvable and distinguishes a difficult control problem from a reward/optimization problem.

Second, test a curriculum: slow incoming shots with no obstacles; randomized one-obstacle shots; then wider speeds and counts. A stage should advance only after held-out interception and scoring thresholds are met. Retain earlier scenarios during later stages to detect forgetting. The current run did not use this curriculum, so its benefit is a proposed hypothesis.

Third, compare the current end-to-end velocity policy against a hierarchical RL formulation. A higher-level policy could output an interception location, desired outgoing direction, and contact speed while a constrained low-level controller executes the strike. This changes the learning problem and may improve sample efficiency, but must be labeled as a new approach rather than retroactively described as the present model. If obstacle avoidance is enforced by a geometric filter, report how often it intervenes and evaluate the unfiltered learned policy separately.

Fourth, run controlled reward ablations and at least three independent seeds. Compare sparse success/contact penalties, current progress shaping, and any revised terminal/timeout balance at equal transition budgets. Select hyperparameters on development layouts and lock a final unseen test set before selection. Track goal rate, collision

rate, interception rate, and timeout rate separately so a more conservative but unproductive policy cannot appear successful through average return alone.

Finally, add hardware-relevant uncertainty only after nominal control is reliable: perturb positions/velocities, emulate dropped frames, insert measured sensor/command delays, randomize friction/restitution, and constrain achievable robot motion. Recurrent policies or observation history may then be appropriate. Validate the state-estimation and actuator models against recorded data before claiming transfer. The current ground-truth simulator makes that gap explicit and measurable.

12. Traceability to the System Design Report

System Design requirement	Evidence in this delivery	Remaining gap
Learned striking and obstacle-aware scoring	Trained state-based PPO, obstacle observations, actual simulator viewer, saved evaluations	Quantitative acceptance depends on measured results; moving obstacles not implemented
At least 80% scoring, ten layouts/twenty attempts, no obstacle collisions	Same episode-count structure with randomized incoming shots	Simulation test is an extension, not full physical-system acceptance
Tracking error at most 2 cm at puck speeds up to 2 m/s	Ground-truth state available for policy tests	No camera tracking error measured
At most 5% tracking dropout over three minutes	No perception dropout in this task	Real detector and sustained video test pending
At least 20 Hz control for 180 seconds	Policy integration interval is 50 ms	Fixed simulation step is not a sustained wall-clock robot test
Mean capture-to-command at most 80 ms; p95 at most 120 ms	CPU normalization plus actor prediction measured	Camera, transfer, filtering, scheduling, and robot dispatch excluded
Half-table workspace and 2 m/s cap	Simulated clipping/projection and speed checks	UR5e joint feasibility and controller limits unvalidated
Emergency stop within 200 ms	No robot command issued by this task	Requires an independent physical acceptance test

The deeper contribution of this report is a fully specified experiment and inspectable evidence, rather than treating project intentions as completed capabilities. The policy can be watched and reproduced, but remaining learning and integration gaps must stay visible.

References

- [1] Schulman et al., *Proximal Policy Optimization Algorithms*, 2017. <https://arxiv.org/abs/1707.06347>
- [2] Stable-Baselines3 2.2.1, PPO documentation and implementation.
<https://stable-baselines3.readthedocs.io/en/v2.2.1/modules/ppo.html>
- [3] TACC, Lonestar6 User Guide, accessed September 24, 2026. <https://docs.tacc.utexas.edu/hpc/lonestar6/>
- [4] TACC, Frontera User Guide, accessed September 24, 2026. <https://docs.tacc.utexas.edu/hpc/frontera/>
- [5] *Supplied System Design Report for a Vision-Based Precision Air Hockey Striker with Obstacle Avoidance*, April 21, 2026, especially Sections 2 and 3 and Appendix B. Source document supplied by the project team; existing performance claims were not independently reproduced here.
- [6] *Supplied TACC Connection and Capabilities Guide*. Used as access-planning context and checked against current system documentation.
- [7] Project source and versioned experiment artifacts, `scripts/precision_striker_env.py`, `scripts/train_precision_striker.py`, `scripts/eval_precision_striker.py`, and `airhockey/sims/airhockey_box2d.py`. These are the primary sources for implementation-specific descriptions and measured results.
- [8] Haarnoja et al., *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*, 2018. <https://arxiv.org/abs/1801.01290>